

Question 1

- In class we presented a dynamic programming solution to the maximum subarray problem with both time and space complexity $\Theta(n)$.
- Suppose that we don't need to find the actual subarray but only the sum of the maximum subarray.
- Write a dynamic programming algorithm that finds the sum of the maximum subarray of an array A of length n that takes $\Theta(1)$ space complexity (time complexity is still $\Theta(n)$).

This is leetcode problem 53.

Since we don't need to find the actual subarray we can use a single variable *last* to store the last value

```
function MaxSubarray(A)  
    n  $\leftarrow$  len(A)  
    last, maxsum  $\leftarrow$  A[0], A[0]  
    for i  $\leftarrow$  1 to n do  
        | last  $\leftarrow$   $\max(A[i], last + A[i])$   
        | if last > maxsum then  
        | | maxsum  $\leftarrow$  last  
    return maxsum
```

Question 2

- A palindrome is a sequence of symbols (a string) P equal to its own reverse.
- For example, "racecar" is a palindrome.
- Given a string S of length n , write a dynamic programming algorithm that finds the **length** of longest palindrome that is a **subsequence** of S
- For example, $S = \text{"cabadc"}$. the longest palindrome subsequence is "cabac" and its length is 5. By contrast, the longest palindrome substring is "aba" (see next question).

Leetcode problem 1143.

Solution

Let x_{ij} be the substring of S from i to j and M_{ij} be the length of the longest palindrome subsequence of x_{ij} . If $S[i] = S[j]$ then $M_{ij} = 2 + M_{i+1,j-1}$. Otherwise, $M_{ij} = \max(M_{i+1,j}, M_{i,j-1})$. Also, we need to take into account that a single symbol is itself a palindrome of size 1.

$$M_{ij} = \begin{cases} 1 & \text{if } i = j \\ 2 + M_{i+1,j-1} & \text{if } S[i] = S[j] \\ \max(M_{i+1,j}, M_{i,j-1}) & \text{otherwise} \end{cases} \quad (1)$$

If we were to solve the problem top down then the recurrence in (1) is sufficient.

To perform the operation bottom up, we need to compute $M_{i+1,j}$ and $M_{i,j-1}$ before M_{ij} . Therefore we iterate i from $n - 1$ down to 0. Note that in this problem the array of characters (string S) is indexed from 0 to $n - 1$.

```
/* M is an n by n matrix initialised to 0 */  
  
for  $i \leftarrow n - 1$  downto 0 do  
     $M[i][i] \leftarrow 1$   
    for  $j \leftarrow i + 1$  to  $n - 1$  do  
        if  $s[i]=s[j]$  then  
             $M[i][j] \leftarrow 2 + M[i + 1][j - 1]$   
        else  
             $M[i][j] \leftarrow \max(M[i + 1][j], M[i][j - 1])$   
    return  $M[0][n - 1]$ 
```

Question 3

- Given a string S of length n you are asked to write a dynamic programming algorithm that finds the longest palindrome that is a **substring** of S .
- For example, for $S = \text{"cabadc"}$, the longest palindrome substring is "aba" .
- Note that in this case your algorithm should return the substring itself and not just the length.

Leetcode problem 5.

- This is similar to the previous problem but with an important distinction.
- S_{ij} is a palindrome iff $S[i] = S[j]$ **and** $S_{i+1,j-1}$ is a palindrome.
- Let $M[i][j] = \text{True}$ iff S_{ij} is a palindrome, and *False* otherwise.
The recursion is

$$M[i][j] = \begin{cases} \text{True} & \text{if } (S[i] = S[j]) \wedge M[i+1][j-1] \\ \text{False} & \text{otherwise} \end{cases}$$

In the algorithm below we use Python slicing convention. So $S[i:j]$ is the substring of S from i to $j-1$.

```
/* M is an n by n matrix initialised to 0 */
```

```
start, end  $\leftarrow$  0, 0
```

```
maxp  $\leftarrow$  1
```

```
for  $i \leftarrow n - 1$  downto 0 do
```

```
     $M[i][i] \leftarrow 1$ 
```

```
    for  $j \leftarrow i + 1$  to  $n - 1$  do
```

```
        if  $S[i] = S[j] \wedge (i = j - 1 \vee M[i + 1][j - 1])$  then
```

```
             $M[i][j] \leftarrow \text{True}$ 
```

```
            if  $j - i + 1 > \text{maxp}$  then
```

```
                maxp  $\leftarrow j - i + 1$ 
```

```
                start, end  $\leftarrow i, j$ 
```

```
            else
```

```
                 $M[i][j] \leftarrow \text{False}$ 
```

```
    return s[start : end + 1]
```


Question 4

Let $A = \{A_1, \dots, A_n\}$ be a set of n integers. Give an algorithm to determine if A has a partition with equal sums. i.e., $\exists X, Y \subseteq A$ such that :

- 1 $X \cup Y = A$
- 2 $X \cap Y = \emptyset$
- 3 $\sum_{x \in X} x = \sum_{y \in Y} y$

Leetcode problem 416.

- Let $S_1 = \sum_{x \in X} x$, $S_2 = \sum_{y \in Y} y$, and $\sigma = \sum_{i=1}^n A_i$.
- Since $S_1 = S_2$ and $X \cup Y = A$ then $S_1 + S_2 = 2S_1 = \sigma$.
- So we need to find a subset of $X \subseteq A$ that sums to $S_1 = \sigma/2$.
- We can use the subsetsum algorithm, covered in class to find such a subset.
- Let $SS(A, k)$ be an implementation of the subsetsum.
- The following algorithm solves the problem

function HasPartition(A)

$\sigma \leftarrow 0$

for $i \leftarrow 1$ **to** n **do**

$\sigma \leftarrow \sigma + A[i]$

if $\sigma \bmod 2 \neq 0$ **then**

return False

$r \leftarrow \text{SubsetSum}(A, \sigma/2)$

if r **then**

return True

return False

Question 5

Suppose that in the previous problem A does not have a partition with equal sums. Give an algorithm to find the partition that minimizes the difference between the sums of the two sets. In other words, find X, Y such that:

- 1 $X \cup Y = A$
- 2 $X \cap Y = \emptyset$
- 3 $\sum_{x \in X} x - \sum_{y \in Y} y$ is minimized.

Let $\sigma = \sum_{x \in A} x$ be the sum of all elements of A . Let $p(i, j)$ be true if the first i elements of A have a subset whose sum is j . Compute $p(i, j)$ for all $1 \leq i \leq n$ and $0 \leq j \leq \sigma/2$. The solution is

$$\min_j \{ \sigma/2 - j \mid p(n, \sigma/2 - j) = \text{True} \}$$

We use the dynamic programming algorithm to solve the subset sum problem with target equal to $\sigma/2$. As we know the solution is a matrix $p(n+1, \sigma/2+1)$ where $p(i, j)$ is true if the first i elements of A have a subset whose sum is j . We choose the last column in row n that is true.

Example: Consider the set $=\{7, 6, 2, 8, 1\}$ whose sum is 24 so $\sigma/2 = 12$. The solution to the subset problem gives

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0	0	0
2	1	0	0	0	0	0	1	1	0	0	0	0	0
3	1	0	1	0	0	0	1	1	1	1	0	0	0
4	1	0	1	0	0	0	1	1	1	1	1	0	0
5	1	1	1	1	0	0	1	1	1	1	1	1	0

From the above matrix we can see that the "best" we can do is to have a subset that sums to 11. The corresponding solution is $X = \{1, 8, 2\}$, the second part of the partition would be $Y = \{7, 6\}$ which sums to 13.

Question 6

Sequence alignment is the problem of arranging sequence of symbols, e.g. DNA, to find maximum matching. For example *ACTGGATT* and *ACGGGTATG* can be aligned, by introducing gaps, as follows:

ACTGG – ATT
ACGGGTATG

The alignment is optimized by minimizing the "cost". The total cost is the sum of individual costs where each dash incurs a cost of α_{gap} and a mismatch a cost of α (matching symbols have a cost of 0). Assuming that $\alpha_{gap} = \alpha = 1$ then the cost of the above alignment is 3 (T with G then T with gap and the last T with G). Write a dynamic programming algorithm to find the optimal alignment of two strings X and Y .

solution

Given strings $X = x_1 \dots x_n$ and $Y = y_1 \dots y_m$ let X_i be the prefix of X of size i , i.e. $X_i = x_1 \dots x_i$. Let $opt(X_n, Y_m)$ be the optimal solution for prefixes $X_n = X$ and $Y_m = Y$. As before we start at the "end" of the strings and move backwards. If $x_n = y_m$ then the cost is zero. If $x_n \neq y_m$ we have 3 options

- 1 Keep the mismatch and incur a cost α
- 2 Extend X by a dash and match y_m with the dash for a cost of α_{gap}
- 3 Extend Y by a dash and match x_n with the dash for a cost of α_{gap}

We choose the smallest cost among all three choices

$$opt(X_i, Y_j) = \min \begin{cases} \alpha + opt(X_{i-1}, Y_{j-1}) \\ \alpha_{gap} + opt(X_{i-1}, Y_j) \\ \alpha_{gap} + opt(X_i, Y_{j-1}) \end{cases}$$

Before we implement the algorithm we need to determine the value of the base cases: $opt(X_i, 0)$ and $opt(0, Y_i)$. Note that when one string is empty we need i gaps to match the other so

$$opt(X_i, 0) = opt(0, Y_i) = i \cdot \alpha_{gap}$$

Consider the two strings $X = "CG"$ and $Y = "CA"$ with the cost of a gap $\alpha_g=3$ and cost of mismatch $\alpha = 7$. The solution obtained from the table below is $CG-$ and $C-A$.

$X \backslash Y$	""	C	CA
""	0	3	6
C	3	$\nwarrow 0$	3
CG	6	$\uparrow 3$	$\leftarrow 6$